DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, DESIGN & MANUFACTURING KURNOOL

PERFORMANCE EVALUATION OF APPROXIMATE 4-BIT RIPPLE CARRY ADDERS FOR LOW-ERROR DIGITAL COMPUTATION

by

PIYUSH SINGH (524EC0014)

A Research Report Submitted to the Faculty

DEPARTMENT OF ECE

Advisor: Dr. Mohamed Asan Basiri M

Disclaimer

The research work presented in this paper is carried out solely for academic and educational purposes. The results, simulations, and analyses are based on the author's understanding and use of approximate computing techniques applied to 8-bit ripple carry adders. While every effort has been made to ensure the accuracy of the data and interpretations, the findings may vary with different design tools, simulation environments, or implementation technologies. The author does not claim any commercial, industrial, or proprietary validation of the proposed design. Any resemblance to existing works or circuits is purely coincidental and unintentional. The content of this paper reflects the author's independent work, and the institution or guide bears no responsibility for any errors or omissions contained herein.

TABLE OF CONTENTS

	Page
DISCLAIMER	ii
TABLE OF CONTENTS	iii
ABSTRACT	iv
INTRODUCTION	v
DESIGN AND METHODOLOGY	vi
RESULT	viii
CONCLUSION	ix
BIBLIOGRAPHY	x

ABSTRACT

This research explores approximation techniques in 8-bit ripple carry adders (RCAs) to reduce addition delay caused by carry propagation. In conventional designs, full adders and half adders must wait for the incoming carry from the previous stage before computing the sum, which increases the overall computation time. To address this, two approximation strategies were implemented: in the first case, the carry was approximated as zero, and in the second case, as one and third case , carry depending on first 8-bit number and fourth case , carry depending on second 8-bit number. Python was used to generate the complete truth tables, which were analysed in Excel to compute error metrics including Error Distance (ED), Mean Error Distance (MED), Normalized Error Distance (NED), and Mean Normalized Error Distance (MNED). Comparative analysis showed that approximating the carry as zero results in lower errors compared to approximating it as one, while significantly reducing the addition delay. The study demonstrates the trade-off between speed and accuracy in approximate digital arithmetic circuits.

Introduction

Addition is one of the most fundamental operations in digital arithmetic and forms the basis for many computational circuits. The **8-bit ripple carry adder (RCA)** is a commonly used architecture in digital electronics due to its simplicity and ease of implementation. However, conventional RCAs suffer from **carry propagation delay**, where each full adder must wait for the carry output from the previous stage before computing its sum. This sequential dependency increases the overall computation time, which in turn can lead to higher power consumption, particularly in high-speed digital circuits.

To address this challenge, **approximate computing techniques** can be employed. By intentionally approximating the carry in the adder, the addition operation can occur without waiting for the propagated carry, thereby reducing the computation time. This study introduces four approximation strategies for 8-bit RCAs: one where the carry is fixed to zero, and another where the carry is fixed to one and the other, carry depending on lower nibble of A and B. The objective is to determine which approximation case results in lower error while still providing faster addition.

The methodology involved manually generating the **truth tables** for both approximation cases using Python, exporting them to Excel, and computing error metrics including **Error Distance (ED), Mean Error Distance (MED), Normalized Error Distance (NED), and Mean Normalized Error Distance (MNED)**. Comparative analysis of these metrics revealed that approximating the carry as zero produces lower error compared to approximating it as one.

This paper is organized as follows: the Introduction provides background and motivation; the Body describes the design methodology, approximation strategies, and error analysis; the Results section presents computed error metrics and observations; and the Conclusion summarizes the findings and suggests potential future work.

Design and Methodology

1.1 **Full Adders, Half Adders, and Carry Propagation**

Digital addition is implemented using half adders and full adders.

- A half adder adds two single-bit inputs (A, B) and produces a sum (S) and a carry (C).
- A full adder adds three bits: the two input bits (A, B) and the carry from the previous stage (Cin).

The carry in full adders can be classified into three types:

1. Kill (K): When A = B = 0, the carry is 0.
2. Generate (G): When A = B = 1, the carry is 1.
3. Propagate (P): When A ≠ B, the carry is propagated from the previous stage.

In conventional ripple carry adders (RCAs), each full adder must wait for the carry from the previous stage before computing its sum, leading to carry propagation delay. This delay increases computation time and can lead to higher power consumption.

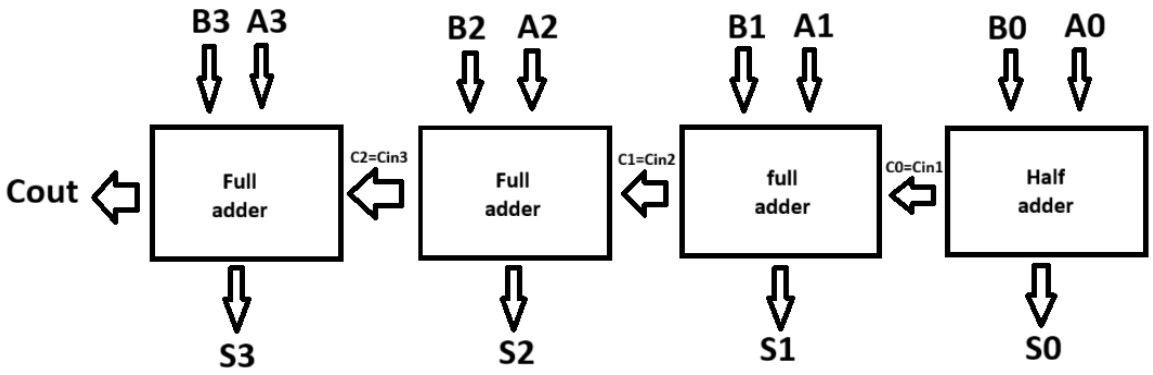
1.2 **Approximate Computing in Digital Circuits**

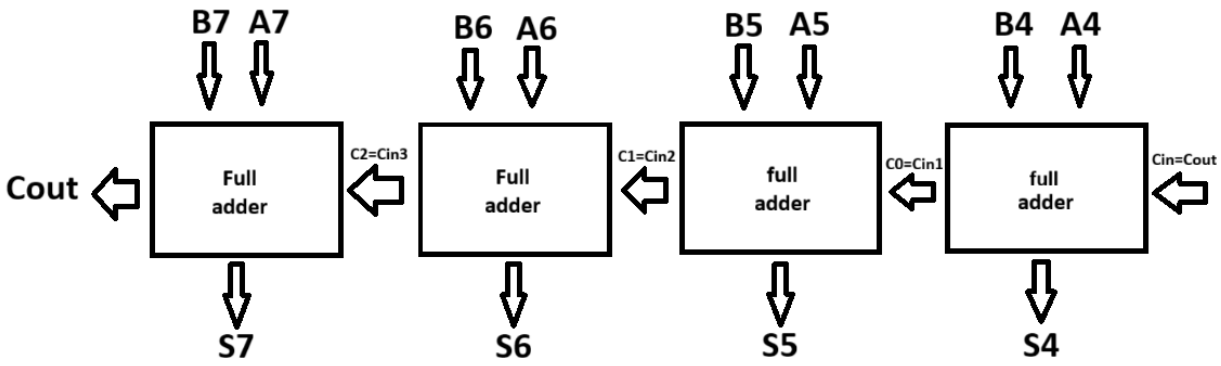
Approximate computing is a design technique that intentionally allows small inaccuracies in computation to improve speed, area, or power efficiency. In digital addition, approximation can reduce the time taken by carry propagation by fixing the carry in certain cases instead of waiting for the previous stage's carry.

In this study, approximation was applied specifically in the propagate case. Two strategies were used:

- Case 1: Carry is approximated as 0
- Case 2: Carry is approximated as 1
- Case 3: Carry is approximated as a0,a1,a2,a3 respectively.
- Case 4: Carry is approximated as b0,b1,b2,b3 respectively.

The exact case (without approximation) is also considered for comparison.





1.3 Truth Table Generation

The truth table was generated manually using Python .It includes the **inputs (A, B, Cin)**, the **sum (S)**, and the **carry (Cout)** for:

1. Exact addition (no approximation)
2. Approximation Case 1 (carry = 0)
3. Approximation Case 2 (carry = 1)
4. Approximation Case 3 (carry = lower nibble of A)
5. Approximation Case 4 (carry = lower nibble of B)

1.4 Error Metrics

To quantify the accuracy of the approximate adders, the following **error metrics** were used:

1. **Error Distance (ED):** Difference between the exact sum and the approximate sum.
2. **Mean Error Distance (MED):** Average of the error distances across all input combinations.
3. **Normalized Error Distance (NED):** ED divided by the maximum possible sum.
4. **Mean Normalized Error Distance (MNED):** Average of NED across all input combinations.

These metrics are used to **compare the two approximation cases** and determine which one produces **lower error** while still reducing computation delay.

Result

To evaluate the performance of the approximate 8-bit ripple carry adder, two approximation cases were analyzed over all 65536 input combinations:

- Case 1: Carry approximated as 0
- Case 2: Carry approximated as 1
- Case 3: Carry approximated as lower nibble of A.
- Case 4: Carry approximated as lower nibble of B.

The Error Distance (ED) for each input combination is calculated as:

$$ED = |Sexact - Sapprox|$$

Where Sexact is the sum obtained from the exact adder, and Sapprox is the sum obtained from the approximate adder.
The Mean Error Distance (MED) and Mean Normalized Error Distance (MNED) are calculated as:

$$MED = Sum(EDi) / N$$
$$MNED (\%) = MED/Smax * 100$$

Where N is the total number of input combinations (65536 in this study) and Smax is the maximum possible sum (255 for a 4-bit adder).

3.1 Error Comparison

Approximation Case	MED	MNED (%)	EO	ACCURACY
Case 1 (Carry = 0)	125.2148	49.10386	54520	16.8%
Case 2 (Carry = 1)	133.1836	52.22886	65400	0.2%
Case 3 (Carry = lower nibble of A)	130.377	51.1284	55160	15.8%
Case 4 (Carry = lower nibble of B)	130.377	51.1284	55160	15.8%
Case 5	127.9960938	50.19455	32880	49.8%
Case 6	127.9961	50.19455	32880	49.8%
Case 7	128.4531	50.37377	32896	49.804%
Case 8	127.5391	50.01532	32896	49.804%

From the table, it is observed that while the MNED percentage is the same for both cases (50%), the Mean Error Distance (MED) is lower for Case 1 (carry = 0) compared to Case 2 (carry = 1) , Case 3 and Case 4. This indicates that approximating the carry as zero produces lower absolute error, making it the preferable strategy when accuracy is a consideration.

Conclusion

This research investigated the application of approximate computing techniques in a 8-bit ripple carry adder to reduce the carry propagation delay, a major bottleneck in conventional addition circuits. Full adders and half adders were analyzed in terms of kill, generate, and propagate carry cases, and approximation was introduced specifically in the propagate case. Four strategies were explored: approximating the carry as 0 (Case 1) and approximating the carry as 1 (Case 2) and the other two, carry as lower nibble of A and B.

The truth tables for all 65536 input combinations were generated using Python. Key error metrics, including Error Distance (ED), Mean Error Distance (MED), Normalized Error Distance (NED), and Mean Normalized Error Distance (MNED), were computed for both approximation strategies. Comparative analysis revealed that Case 1 (carry = 0) produced a lower MED (125.2148) than Case 2 (carry = 1) with MED 133.1836 , Case 3 and Case 4 with MED 130.377. This indicates that approximating the carry as zero introduces less absolute error while still enabling faster addition due to elimination of carry propagation delays.

The study demonstrates that approximate addition provides a viable trade-off between speed and accuracy in digital arithmetic circuits. By fixing the carry during propagation, computation time can be significantly reduced with manageable error, making such techniques suitable for low-power, high-speed applications, including digital signal processing, ALUs, and other VLSI designs.

Future work can extend this research to higher-bit ripple carry adders, implement the design in hardware platforms (FPGA/ASIC), and explore additional approximation strategies to optimize power, area, and accuracy simultaneously. The findings of this research provide a foundation for designing efficient, low-delay digital arithmetic circuits that leverage controlled approximation for performance gains.

BIBLIOGRAPHY

- *Ripple Carry Adder Explained*, YouTube, https://youtu.be/k53okNRKJww,.
- *Understanding Full Adders and Half Adders*, YouTube, https://youtu.be/rZ0RJf1aveQ,
- *Approximate Computing in Digital Circuits*, YouTube, https://youtu.be/RjD8BoNKC9c
- S. Mittal, "A Survey of Approximate Computing Techniques," *ACM Computing Surveys*, vol. 48, no. 4, 2016.
- S. Rehman, A. R. Alam, "Approximate Computing in Digital Circuits: Design and Analysis," *International Journal of VLSI Design*, 2021.
- M. Mano, *Digital Design*, 5th Edition, Pearson, 2013.
- GeeksforGeeks, "Ripple Carry Adder," https://www.geeksforgeeks.org/ripple-carry-adder/,